Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19-03-2026**	**Enrolment No:92301733024**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models. Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19-03-2026**	**Enrolment No:92301733024**

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus
2. Perform TF vectorization of the corpus

Program (Code):

```
sentences = [
   "I am studying Machine Learning this semester.",
   "I completed the Networking course last year.",

   "Tiger is the national animal of India.",
   "India has a rich cultural heritage.",
   "Rajasthan is a famous state in India.",

   "I enjoy playing football with my friends.",
   "I like watching basketball matches."
]

import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer #TF-BoW
from sklearn.feature_extraction.text import TfidfVectorizer #TF-IDF
from sklearn.metrics.pairwise import cosine_similarity

#TF-BowEmbedding
tf_vectorizer=CountVectorizer()
tf_emb=tf_vectorizer.fit_transform(sentences)

#similarity matrix using cosine_similarity
tf_sim=cosine_similarity(tf_emb)
```

tf_sim

```
array([[1.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 1.        , 0.15430335, 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 0.15430335, 1.        , 0.16903085, 0.3086067 ,
        0.        , 0.        ],
       [0.        , 0.        , 0.16903085, 1.        , 0.18257419,
        0.        , 0.        ],
       [0.        , 0.        , 0.3086067 , 0.18257419, 1.        ,
        0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        1.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 1.        ]])
```

```
#TF-BowEmbedding
tfidf_vectorizer=TfidfVectorizer()
tfidf_emb=tfidf_vectorizer.fit_transform(sentences)

tfidf_sim=cosine_similarity(tfidf_emb)

tfidf_sim
```

```
array([[1.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 1.        , 0.1191192 , 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 0.1191192 , 1.        , 0.09781937, 0.21578311,
        0.        , 0.        ],
       [0.        , 0.        , 0.09781937, 1.        , 0.10410758,
        0.        , 0.        ],
       [0.        , 0.        , 0.21578311, 0.10410758, 1.        ,
        0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        1.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 1.        ]])
```

```
#Recommend the most matching/similar sentence
def doc_sim(tfidf_sim,sentence):
  for i in range (len(sentences)):
    sim=tfidf_sim[i].copy()
    sim[i]=-1
    top_index=np.argmax(sim)
    print(sentences[i],"->",sentences[top_index])
```

```
doc_sim(tfidf_sim,sentences)
```

```
I am studying Machine Learning this semester. -> I completed the Networking course last year.
I completed the Networking course last year. -> Tiger is the national animal of India.
Tiger is the national animal of India. -> Rajasthan is a famous state in India.
India has a rich cultural heritage. -> Rajasthan is a famous state in India.
Rajasthan is a famous state in India. -> Tiger is the national animal of India.
I enjoy playing football with my friends. -> I am studying Machine Learning this semester.
I like watching basketball matches. -> I am studying Machine Learning this semester.
```

```
#SBERT for meaning level similarities

from sentence_transformers import SentenceTransformer
sbert_model = SentenceTransformer('all-MiniLM-L6-v2')

sbert_embeddings = sbert_model.encode(sentences)
sbert_sim = cosine_similarity(sbert_embeddings)

print("SBERT Similarity Matrix:\n")
print(sbert_sim)

def sbert_doc_sim(sim_matrix, sentences):
    print("\nSBERT Semantic Recommendations:\n")
    for i in range(len(sentences)):
        sim = sim_matrix[i].copy()
        sim[i] = -1
        top_index = np.argmax(sim)
        print(sentences[i], "->", sentences[top_index])

sbert_doc_sim(sbert_sim, sentences)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19-03-2026**	**Enrolment No:92301733024**

```
SBERT Similarity Matrix:

[[ 1.          0.27516615  0.06015403 -0.02255641  0.04702425  0.06591374
   0.10194391]
 [ 0.27516615  1.          0.01572408 -0.02594151  0.05585686  0.10834689
   0.08604154]
 [ 0.06015403  0.01572408  0.9999999   0.40113652  0.3005461   0.03677317
   0.03982799]
 [-0.02255641 -0.02594151  0.40113652  1.0000004   0.5224977   0.08154863
   0.10931155]
 [ 0.04702425  0.05585686  0.3005461   0.5224977   1.0000001   0.06853396
   0.13030206]
 [ 0.06591374  0.10834689  0.03677317  0.08154863  0.06853396  1.0000001
   0.55862975]
 [ 0.10194391  0.08604154  0.03982799  0.10931155  0.13030206  0.55862975
   1.0000002 ]]

SBERT Semantic Recommendations:

I am studying Machine Learning this semester. -> I completed the Networking course last year.
I completed the Networking course last year. -> I am studying Machine Learning this semester.
Tiger is the national animal of India. -> India has a rich cultural heritage.
India has a rich cultural heritage. -> Rajasthan is a famous state in India.
Rajasthan is a famous state in India. -> India has a rich cultural heritage.
I enjoy playing football with my friends. -> I like watching basketball matches.
I like watching basketball matches. -> I enjoy playing football with my friends.
```

Observation:

- The corpus is converted into numerical vectors using BoW and TF-IDF.
- Similarity between sentences is calculated using cosine similarity.
- Sentences with similar words have higher similarity values.
- SBERT gives better results as it considers meaning of sentence

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19-03-2026**	**Enrolment No:92301733024**

Post Lab Exercise:

Take three documents and find similarity using

a. BoW vectorization
b. TF Vectorization
c. TF-IDF Vectorization

Comment over the answer obtained in each of the case.

a. BoW vectorization

BoW considers only word count.
It ignores semantic meaning of words.
Similarity is based on common words only.

b. TF vectorization

TF gives importance to frequently occurring words.
It improves similarity compared to BoW.
Still, it does not consider context or meaning.

c. TF-IDF vectorization

TF-IDF reduces the weight of common words.
It gives more importance to rare and meaningful words.
It provides better similarity results than BoW and TF.